

15

Cyber Security
Codec Technologies India
Internship 2026

Email Security Gateway

Six-layer .eml filter with real DMARC alignment

AT A GLANCE

UCI SMS (real)

ML corpus

0.9392

ML F1

0.9839

ML accuracy

8 (5 rejected)

Samples

6

Detection layers

23 / 23

Tests re-run

2 / 2

Benign allowed

VERDICT

TESTS 23/23

RE-RUN HERE

REAL DATASET

DEFENSIVE ONLY

API STUB (NO CALLS)

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

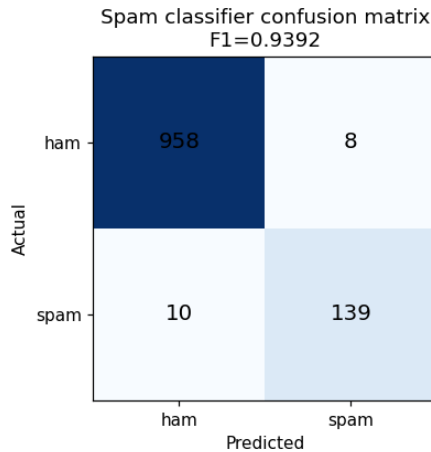
Objective & brief mapping

The brief asked for an email security gateway. I built a defensive filter that ingests raw .eml, runs each message through six independent detection layers, and produces an ALLOW/QUARANTINE/REJECT verdict with reasons. I made the auth layer honest about spoofing: it re-computes DMARC alignment itself rather than trusting a dmarc=pass header.

Architecture

Parsing normalises the .eml; then SPF/DKIM/DMARC alignment, sender reputation, URL extraction + lexical scoring, attachment inspection (macros, double extensions, zip bombs), header anomalies, and a TF-IDF + LinearSVC ML classifier. The verdict engine folds all signals into one weighted risk score.

```
.eml -> parser -> [auth align | reputation | url scan |
  attachments | header anomalies | ML] -> verdict engine
risk>=0.75 REJECT, >=0.35 QUARANTINE, else ALLOW
```



ML spam/phish confusion matrix (results/confusion_matrix.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python main.py # generate data if missing, process mailbox (<3 s)
python3 unittest-style runner over tests/ # 23 tests
```

The artefacts each figure and table above are drawn from:

- **metrics.json** — ML P/R/F1 + confusion
- **verdicts.csv** — per-message verdicts
- **confusion_matrix.png** — ML plot
- **quarantine_report.md** — analyst report

Method & implementation

DMARC passes only if SPF or DKIM passes AND its domain aligns (relaxed = same registrable domain) with From. A message with spf=pass for the attacker's bounce domain but a spoofed From correctly fails. External reputation (VirusTotal/urlscan) is a documented stub that makes **no** network calls.

Results

ML from results/metrics.json (real UCI SMS Spam Collection, 5,574 msgs, 80/20 split); verdicts from verdicts.csv.

Item	Metric	Value
ML classifier	F1	0.9392
ML classifier	Accuracy	0.9839
ML confusion	[[tn,fp],[fn,tp]]	[[958,8],[10,139]]
Verdicts	totals	2 ALLOW, 1 QUARANTINE, 5 REJECT
phish_credential.eml	REJECT	risk 2.02
malware_double_ext.eml	REJECT (hard)	scan.pdf.exe

Six detection layers

Layer	Checks	Weight
auth alignment	DMARC re-computed	hard signal
reputation / URL	allow-deny + 14 features	medium
attachments	macro/double-ext/zip-bomb	hard reject
headers / ML	spooft + TF-IDF SVC	weighted

Verification

KEY TAKEAWAY A transparent gateway that re-computes DMARC alignment itself (catching valid-SPF/spoofed-From phishing) and stopped every phishing/malware sample while allowing both benign.

15

Cyber Security
Codec Technologies India
Internship 2026

Email Security Gateway

Six-layer .eml filter with real
DMARC alignment

AT A GLANCE

UCI SMS (real)

ML corpus

0.9392

ML F1

0.9839

ML accuracy

8 (5 rejected)

Samples

6

Detection layers

23 / 23

Tests re-run

2 / 2

Benign allowed

VERDICT

TESTS 23/23

RE-RUN HERE

REAL DATASET

DEFENSIVE ONLY

API STUB (NO CALLS)

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Re-ran here, 23/23 (~1.6 s) — the second-largest suite. It includes the "SPF passes but unaligned → DMARC fails" spoof case and ML F1 > 0.90.

Test group	Checks	Result
Auth alignment	spoof case fails, DKIM-only aligns	PASS
Attachments/URLs	double-ext, macro, zip-bomb, punycode	PASS
Pipeline/ML	benign ALLOW, phish/malware REJECT, F1>0.90	PASS
Total	23 tests	23/23

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

Real ML corpus and detection logic; the external reputation API is a documented stub that never calls out.

Ran here	Real integration	Note
TF-IDF + LinearSVC	commercial secure email GW	transparent equivalent
DMARC alignment re-computed	upstream MTA auth	catches spoofed From
URL reputation stub	VirusTotal / urlscan.io	no network calls made

Ethics, data integrity & declared substitutions

Defensive and educational. It detects and scores inbound mail; it never sends mail, never phishes, and never contacts a real third party. **Data integrity:** the ML corpus is the real public UCI SMS Spam Collection (fetched with curl); every .eml, the reputation list and the URL blocklist are synthetic on fictional Acme Corp / .example domains, and the "malware" attachments are inert text stand-ins. **Declared substitution:** the external-API integration is a stub that makes no calls.

Learnings

Re-implementing DMARC alignment rather than reading a header taught me where real-world spoofing hides: an attacker can get spf=pass for a domain they own while forging the visible From. Folding six independent signals into one weighted verdict — with hard-reject overrides for zip bombs and double extensions — is the layered-defence pattern I will reuse.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Alignment relies on the MTA's Authentication-Results stamp; no live SPF DNS / DKIM crypto.
- The public-suffix list is a tiny hard-coded subset (production: full Mozilla PSL).
- The ML corpus is SMS-length text; an email corpus would generalise better to long bodies.
- Lexical URL scoring is heuristic; production pairs it with live reputation feeds.
- ML trained on the real UCI SMS Spam Collection (5,574 msgs), F1 0.9392.
- Verdicts on 8 samples: 2 ALLOW, 1 QUARANTINE, 5 REJECT.
- Zip bombs and double extensions are immediate hard REJECTs.
- All .eml samples and the malware stand-ins are synthetic and inert.
- ML trained on real UCI SMS Spam (4,827 ham / 747 spam).
- Held-out F1 0.9392, precision 0.9456, recall 0.9329, accuracy 0.9839.
- Confusion [[958,8],[10,139]] on 1,115 held-out messages.
- Verdicts: 2 ALLOW, 1 QUARANTINE, 5 REJECT on 8 samples.
- phish_credential REJECT at risk 2.02; macro .xslm QUARANTINE.
- DMARC passes only if SPF or DKIM aligns with From.
- External reputation is a stub that makes no network calls.
- References: RFC 7208/6376/7489, UCI, VirusTotal, urlscan.io.
- Reference: RFC 7208 (SPF); RFC 6376 (DKIM); RFC 7489 (DMARC).
- Reference: UCI SMS Spam Collection; SpamAssassin public corpus.
- Reference: VirusTotal API v3; urlscan.io; RFC 2606.